1) **Navesti bar 5 HTML tagova za formatiranje i isticanje teksta i objasniti.**

Tagovi za formatiranje teksta

- <b>
  - bold, **podebljano**
- <i>
  - italic, *iskošeno*
- <u>
  - underline, <u>podvučeno</u>
- <sub>
  - Subscript, indeks elementa, $_{dole}$
- <sup>
  - superscript , stepen elementa, ^{gore}

Tagovi za isticanje teksta

- <big>
  - veća slova (nije podržan u HTML5)
- <small>
  - manja slova
-
  - emphasized, naglašeno, blago ukošeno
-
  - naglašeno, važno podebljano

2) Kako se prati sesija u ASP.NET?

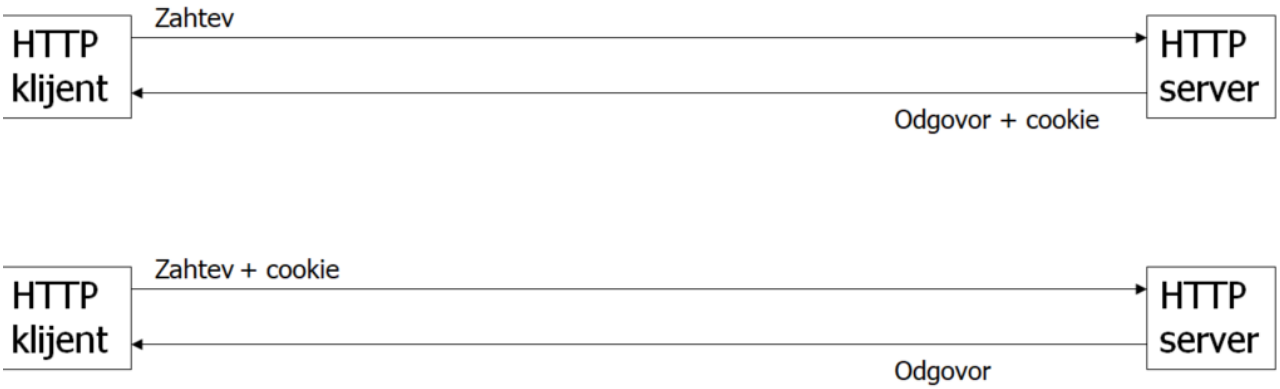
Praćenje sesije korisnika 1/3

- HTTP protokol ne prati sesiju – veza klijenta i servera se zatvara po isporuci resursa.
- Koristi se *cookie* mehanizam:
  - Server šalje *cookie* klijentu u okviru http response
  - klijent čuva primljeni *cookie* i šalje ga uz svaki http request.

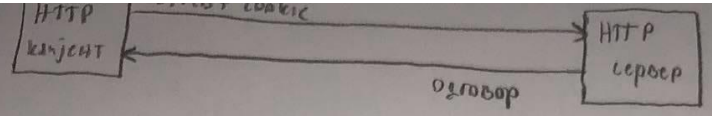
3/45

Praćenje sesije korisnika 2/3

- kada se koristi cookie mehanizam



3) Navesti primer C# koda za ispis promenljive pod nazivom ime, Sta treba da uradimo ako imamo promenljivu imena koja predstavlja listu vrednosti?
4) Navesti i objasniti 4 nacina prenosa parametara forme iz kontrolera u pogled. (ovo je sa FB vrv je greska, vrv treba Prenos podataka iz kontrolera u pogled)



3) Навести пример C# кода за испис променљиве под називом ime. Шта треба да урадимо ако имамо променљиву imena која представља листу вредности?

4) Навести и објаснити 4 начина преноса параметара форме из контролера у поглед. (То је са FB вр. прежа, треба Пренос података из контролера у поглед)

1) ViewData - приступ преко кључа

- контролер: ViewData["student"] = "Aleksa";
- поглед: <h2> @ViewData["student"] </h2>
- захтева кастовање ако су питању комплексни типови података
- ако дође до редирекције тада његова вредност постаје null

Контролер: var Numbers = new List<int> {1,2,3};
ViewData["numbers"] = Numbers;

=> поглед:

@foreach (var number in (List<int>) ViewData["numbers"]){
<h2> number </h2>

2) ViewBag - приступ као код пропертија

- контролер: ViewBag.ime = "Aleksa";
- поглед: <h2> @ViewBag.ime <h2>
- Нема кастовања ако су у питању комплексни типови података
- ако дође до редиректиње тада његова вредност постаје null

3) TempData - приступ преко кључа

- омогућује размену података између више HTTP request-ова, све док се први пут не оцита у оквиру погледа, тада се TempData објекат брише
- користи се код редиректиње између акција
- контролер: TempData["ime"] = "Aleksa";
- поглед: <h2> @TempData["ime"] </h2>

4) Model

- контролер: List<Product> proizvodi = new List<Products>();
  return View(proizvodi);
- поглед: @model IEnumerable<Product>
  @foreach (Product p in Model) {
  <h2> @p.ime </h2>
  }

5) Шта је ASP.NET Web Api?

- Концепт: - web апликација је Java Script апликација која се извршава на клијенту (br
- web server

5) Sta je ASP.NET web api?

5. ASP.NET WebAPI

- WebAPI se zasniva na drugačijem konceptu:
  - web aplikacija je JavaScript aplikacija, koja se izvršava na klijentu (browser)
  - web server služi da primi i pošalje podatke od/ka web aplikaciji
  - metode C# klasa na serveru se "gađaju" iz klijenta preko URL-a
    - JavaScript upućuje HTTP zahtev ka serveru
    - web server pronalazi odgovarajuću klasu i metodu
    - povratna vrednost metode se vraća u JavaScript kod
    - JavaScript na osnovu primljenih podataka ažurira odgovarajući deo stranice.

6) Sta je JSON, cemu sluzi?

JSON

- JavaScript Object Notation
- Jednostavan format za razmenu podataka predstavljenih pomoću teksta
- Nezavistan od konkretnih tehnologija sa kojima se koristi
- Lako se razume
- Sintaktički identičan kodu za kreiranje objekata u JavaScript-u

- Danas primarni format podataka korišćen za asinhronu komunikaciju između klijenta i servera.

JSON sintaksa

- Nizovi ili kolekcije su predstavljene simbolom '['
- Stringovi su pod navodnicima, primitivni tipovi nisu
- Vitičastim zagradama obeležavamo blok ili strukturu

JSON - Primer

```
{
"studenti": [
  { “id”:1, "ime":"Pera" , "prezime":"Perić",
    “email”:”pera@gmail.com” },
  { “id”:2, "ime":"Steva" ,
    "prezime":"Stević", “email”:null },
  { “id”:3, "ime":"Jova" , "prezima":"Jović",
    “email”:”jova@gmail.com” }
]
}
```

JSON - Primer

```
{
"firstName": "John",
"lastName": "Smith",
"age": 25,
"address": {
      "streetAddress": "21 2nd Street",
      "city": "New York",
      "state": "NY",
      "postalCode": 10021
      },
"phoneNumbers": [
       { "type": "home", "number": "212 555-1234" },
       { "type": "fax", "number": "646 555-4567" }
      ]
}
```


7) Sta je forma, cemu sluzi, kako se koristi i kako se definise?

Forme u ASP.NET-u

- Podaci se unose preko HTML formi.
- Na *cshtml* stranici se parametrima forme pristupa preko `Request` promenljive:

```
Request["username"]
```

- Izraz `Request["username"]` vraća `null`, ako ne postoji parametar forme pod tim imenom.

Primer

```
@{
MyUser u = new MyUser();
u.username = Request["username"];
u.password = Request["password"];
}
@if (u.login())
{
<p>Uspešno ste se prijavili</p>
}
```

ugradjeniObjekti/ugradjeni.cshtml#forme

8) Data je klasa student sa atributima(string ime, string prezime,AdresaStanovanja adresa),gde je adresa objekat klase AdresaStanovanja koja ima atribute(string ulica, string grad, string postanski kod).Napisati primer json stringa za objekat klase Student.

   iskoristiti gore json primer

9) Kada korisnik izabere neki fajl i posalje ga serveru, sta radi default prenos? (zaokruzivanje)
   multipart/file-data

10) Navesti bar 3 nacina pristupa DOM elementima u JavaScriptu.

Pristup elemenata

- Postoji više načina da se iz JavaScript-a pristupi određenim elementu/elementima forme (npr. tekst polju za unos vrednosti)

1. Način – oslanjajući se na jedinstveni id elementa u okviru dokumenta i funkciju *getElementById.*
   `document.getElementById('textbox_id')`
   primer: **`document.getElementById('polje1');`**

2. Način – oslanjajući se na vrednost atributa class (CSS klase) za elemente i funkciju *getElementsByClassName.* Kako više elemenata može koristiti istu CSS klasu, tako je rezultat pomenute funkcije kolekcija elemenata.
   `document.`
   `getElementsByClassName('class_name')[whole_number]`
   primer: **`document.getElementsByClassName("searchField")[0]`**

Pristup elemenata

3. Način – oslanjajući se na ime elementa i funkciju *getElementsByName.* Kako više elemenata sa istim imenom može postojati u okviru dokumenta, tako je rezultat pomenute funkcije kolekcija elemenata.
   `document.getElementsByName('name')[whole_number]`
   primer:
   `document.getElementsByName("input_field")[0]`

4. Način – oslanjajući se na ime html elementa i funkciju *getElementsByTagName.* Kako više istih elemenata može postojati u okviru dokumenta, tako je rezultat pomenute funkcije kolekcija elemenata.
   `document.getElementsByTagName('tag_name')[whole_number]`
   primer:
   `document.getElementsByTagName("input")[0]`

Pristup elemenata

5. Način – oslanjajući se na sintaksu CSS selektora i funkciju *querySelector*. Funkcija vraća prvi element u dokumentu koji zadovoljava uslov.

```
document.querySelector('selector')
```

primer:
selekcija na osnovu id elementa
```
document.querySelector('#searchTxt')
```
selekcija na osnovu CSS klase
```
document.querySelector('.searchField')
```
selekcija na osnovu imena elementa
```
document.querySelector('input')
```
selekcija na osnovu atributa elementa
```
document.querySelector('[name="naziv"]')
```

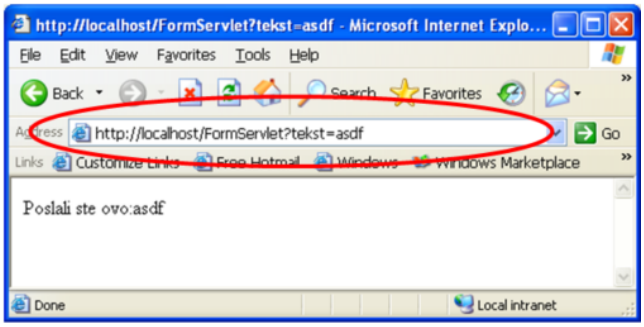
11) Napisati glavnu razliku izmedju GET i POST zahteva kod prenosa parametara forme.

GET i POST zahtevi

Kod GET metode se parametri forme nalaze u heder delu HTTP request poruke

```
GET /FormServlet?tekst=asdf HTTP/1.1
...
```

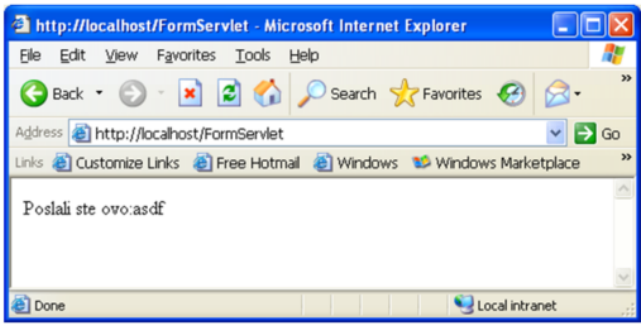
Kod HTTP **GET** metode, posle znaka ? zapisuju se vrednosti URL promenjivih tipa ***ključ=vrednost&ključ=vrednost***...




```
POST /FormServlet HTTP/1.1
...
Content-length: 10
...

tekst=asdf&kljuc=vrednost
```

Kod HTTP **POST** metode se vrednosti parametara forme smeštaju na kraju HTTP zahteva (posle praznog reda), i posle vrednosti parametara nema "\r\n" karaktera na kraju reda



Kod POST metode se parametri forme nalaze u body delu HTTP request poruke

12) Objasniti sta znace sledeci CSS izrazi: a>b(....), a*b(...), a+b(...),a b(...)

- **Odnosi se na tag koji se nalazi unutar drugog taga (nedefinisane dubine – ne mora biti direktan potomak)**
- Primer:

```
a b {...}
```

– važi za b koji je bilo koji potomak od a

```
div p * u {...}
```

– stil se odnosi na <u> tag koji za pretka ima bilo koji tag, koji za svog pretka ima <p> tag, koji za svog pretka ima tag

```
body > p {color: blue}
```

– važi za <p> tag čiji je neposredni roditelj <body> tag

```
a + b {...}
```

– važi za element b koji ima istog pretka kao i element a, i element a se nalazi pre elementa b

13) Navesti primer da se oboji tekst u plavo i podebljaju slova za CSS u JQuery

```
$("p").css("color", "red").hide(1000).show(1000);
$("div").animate({
  left:'250px',
  height:'+=150px',
  width:'+=150px'
});
```

guglaj nisam nista bolje nasao

14) Navedi i objasni koje vrste HTML tagova postoje

- Tag se završava znakom >
- Vrste:
  - složeni (imaju i otvarajući i zatvarajući tag)
  - prazni (prosti) tagovi, gde nema zatvarajućeg taga i sadržaja
    - primer: `<br />`

Prosti – ne sadrze zatvarajuci tag.
Slozeni – sadrze I otvarajuci I zatvarajuci tag.
Osnovni html tagovi su:
<html> - okvir u kom se nalaze svi ostali tagovi
<head> - uokviruje zaglavlje u kom se nalaze informacije o samom dokumentu(opcion je)
<body> - Sve ono što vidimo u prozoru browser-a, tj. sadržaj stranice, nalazi se u telu dokumenta koje uokviruje element
Neki od tagova su:

- <h1> – <h6> – naslovi
- <p> – paragraf
-
 – line break
- <hr /> – horizontal ruler – horizontalna crta
- <!-- komentar -->

15) Navedi i objasni barem 5 HTML tagova koji omogucavaju formatiranje teksta koji je citat ili definicija.

Skraćenice, citati, definicije

- <blockquote>
  - duži citat
- <q>
  - kraći citat
- <cite>
  - citat
- <dfn>
  - za definicije u tekstu

- <b> – bold, podebljano
- <i> – italic, iskošeno
- <u> – underline, podvučeno
- <sub> – Subscript, indeks elementa, dole
- <sup> – superscript , stepen elementa, gore

16) Gde se stilovi ugradjuju za CSS, prioritet preklapanja stilova?

Gde se stilovi ugrađuju?

- Unutar samih HTML elemenata (atribut **style**)
  - inline style
- Upotrebom taga <style> unutar dokumenta (unutar <head> taga)
  - internal style sheet
- Kreiranjem spoljašnje datoteka stilova (.css datoteka)
  - external style sheet
- Ako dođe do preklapanja, prioritet je:
  1. unutar HTML elementa
  2. <style> tag
  3. spoljašnja datoteka stilova
  4. kako čitač prikazuje

17) Razlika izmedju 3 nacina definisanja stila:
p {color:blue} -primenjuje se na svaki html element tipa p
p.PlaviTekst{color:blue} -klasa definisana za html element tipa p
.PlaviTekst{color:blue} -klasa definisana za bilo koji element

18) Opisati ukratko strukturu HTTP zahteva

HTTP zahtev

- Počinje redom:
  METOD /putanja HTTP/verzija
- METOD je:
  - GET,
  - POST, i dr.
- dodatni redovi sadrže atribute oblika:
  Ime: vrednost
- prazan red na kraju
  - Ako je POST zahtev posle praznog reda idu parametri forme

HTTP odgovor

- Počinje redom:
  HTTP/verzija kod tekstualni_opis
- dodatni redovi sadrže atribute:
  Ime: vrednost
- prazan red
- sledi sadržaj datoteke

"200" ; OK

"201" ; Created

"202" ; Accepted

"204" ; No Content

"301" ; Moved Permanently

"302" ; Moved Temporarily

"304" ; Not Modified

"400" ; Bad Request

"401" ; Unauthorized

"403" ; Forbidden

"404" ; Not Found

"500" ; Internal Server Error

"501" ; Not Implemented

"502" ; Bad Gateway

"503" ; Service Unavailable

17) Objasniti razliku između sledećih stilova:

```
p {color: blue}
p.plavi Tekst {color: blue}
.plavi Tekst { color: blue}
```

18) Opisati ukratko okruženje HTTP zahteva.

19) Navesti naziv i značenje reči HTTP zahteva (HTTP verzija 1.1) koji se koristi za slanje na server "Poslati" u sledećoj formi:

```
<form action="http://localhost/Proba" method="GET">
    <input type="text" name="polje1" value="">
    <input type="text" name="polje2" value="test">
    <input type="submit">
</form>
```

20) Kako se prati sesija u ASP.NET okruženju. Dati primer u kodu kako se ona postavlja i proverava.

21) Objasniti predefinisane promenljive Session, Application, Cache i Page.

GET/http://localhost/Proba/1.1

20)Kako se prati sesija u ASP.NET okruzenju? Dati primer u kodu kako se ona postavlja i proverava.

Ima vec gore 2. pitanje ovde je jos samo slika primera u kodu

Praćenje sesije korisnika na ASP.NET stranici

```
@{
        MyUser u = (MyUser)Session["User"];
        if (u == null)
        {
                u = new MyUser();
                Session["User"] = u;
        }
        if (!u.loggedIn) {
          u.username = Request["username"] != null ? Request["username"] : "";
         u.password = Request["password"] != null ? Request["password"] : "";
          u.login();
        }
}
@if (u.loggedIn)
{
        <p>
                Uspešno ste se prijavili:<br />
                Username: <b>@u.username</b><br />
                Password: <b>@u.password</b>
        </p>
}
```

21)Objasniti predefinisane promenljive Session, Application, Cache i Page

• **Session** objekat povezan sa zahtevom i tipa je HttpSessionStateBase. Sadrži asocijativnu mapu objekata koji de pratiti korisnike, na osnovu kukija.

•**Cache** promenljiva se koristi za smeštanje globalnih vrednosti na nivou aplikacije koje imaju privremeni karakter. – Prilikom dodavanja entiteta u Cache omogudeno je postavljanje životnog veka za entitet. Metode koje to omogučuju se add i insert prilikom čega je mogude postaviti apslutni životni vek entiteta (npr. 60 minuta) ili životan vek koji se vezuje za nekorišdene entiteta (npr.10 minuta).

•**Application** promenljiva se koristi za smeštanje globalnih vrednosti na nivou aplikacije koje se mogu inicijalizovati u Global.asax.cs datoteci. Objekti postavljeni u Application nestaju samo ako se web aplikacija ugasi. Preporučuje se za podešavanje globalih promenljivih koji sadrže parametare podešavanja aplikacije. Ne treba je koristiti za skladištenje kolekcija koje sadže veliki broj objekata.

•**Page** - Sadrži asocijativnu mapu objekata zakačenih i vidljivih samo na tekudoj stranici. Mora se prvo definisati da bi se koristio.

22) Sta predstavlja ViewModel u MVC, kada se koristi?

To su C# klase koje imaju za ulogu da objedine više modela, ili onih delova modela koji su relevantni za određeni prikaz.

•Od velike koristi pri validaciji korisničkih unosa sa UI-a

• Može da sadrži podatke od strane različitih izvora

• Može da sadrži polja koja se nalaze isključivo na prikazu

Kreirani objekti ViewModel-a se šalju u pogled, npr. pogled za izmenu proizvoda.

slajdovi:

- Šta ako je za formiranje mog prikaza potrebno više različitih modela?
- Tada treba koristiti ViewMode
- To su C# klase koje imaju za ulogu da objedine više modela ili onih delova modela koji su relevantni za određeni prikaz.
- Od velike koristi pri validaciji korisničkih unosa sa UI-a
- Može da sadrži podatke od strane različitih izvora
- Može da sadrži polja koja se nalaze isključivo na prikazu
- ViewModeli se u strukturi ASP.net MVC projekta prave u zasebnom folderu ViewModels.

23) Pri slanju parametara forme u POST zahtevu, ako se ne salje datoteka, default tip podataka koji se salju na server je smesten u parametru zahteva. Zaokruzi odgovor.
Content-Type : application/x-www-form-urlencoded

24) Na koja 3 nacina se definisu korisnicki definisani objekti u JavaScriptu?

Kreiranje objekta

- Korišćenjem "JSON" sintakse:

```
var osoba = {ime:"pera", prezime:"peric"};
```

- Korišćenjem ključne reči **new** i **object**:

```
var osoba = new Object();
osoba.ime="pera";
osoba.prezime="peric";
```

- Korišćenjem konstruktora:

```
function osoba(ime, prezime) {
   this.ime=ime;
   this.prezime=prezime;
}
var osoba = new osoba("pera", "peric");
osoba.ime="pera1";
osoba.prezime="peric1";
osoba.adresa="adr1";
```

25) Sta je JQuery, namena, kako se poziva?

jQuery

- JavaScript biblioteka
- Osmišljena da pojednostavi programiranje
- Pojednostavljuje:
  - Jednostavan pristup elementima Web stranice
  - Izmena izgleda Web stranice
  - Izmena sadržaja Web stranice
  - Interakcija sa korisnikom
  - Animacije
  - Dobavljanje sadržaja sa servera bez ponovnog osvežavanja stranice
  - Uobičajene JavaScript komande

- Za početak, mora da se uključi u stranicu <script> tagom koji učitava jQuery.js datoteku

- Sve počinje

```
$(document).ready(function() {
   ...
});
```

funkcijom, unutar <script> taga u <body> sekciji.

26) Opet onaj JSON oblik za studenta i adresu stanovanja

27) 5 tagova za formatiranje teksta - vec bilo
28) CSS skracenica i definicija

- HTML dokumenti se zatrpavaju dodatnim tagovima i atributima za formatiranje izgleda
- CSS rešava ovaj problem
  - izgled se podešava definicijom stila
  - definicije stila se ne moraju nalaziti u istom dokumentu

CSS

- **C**ascading **S**tyle **S**heets
  - može više definicija za isti element da se preklapa (cascade)
- HTML bi trebalo da se koristi za opis strukture dokumenta
  - HTML ima default način prikaza elemenata
- Vizuelna definicija (izgled) HTML stranica se prepušta stilovima (CSS):
  - stilovi se definišu za elemente HTML-a (tagove)
  - stilovi definišu izgled elemenata (boja, font, pozadinska boja, itd.)
  - jedan stil može da se koristi i za više HTML datoteka

29) Kako se pise funkcija u cshtml (2 nacina)?

Funkcija u bloku

```
@{
  var poslednji = new Func<string[], string>((braca) =>
   braca.Last());
}
```

- Generic delegat

```
public delegate TResult Func<in TArg0, out TResult>(TArg0 arg0)
```

- Enkapsulira metodu koja ima 1 parametar tipa TArg0 i vraća 1 vrednost koja je tipa Tresult
  - Func delegat se često primenjuje za enkpsulaciju lambda izraza koji se primenjuju na svaki element iz kolekcije ulaznih podataka

Blok funkcija

Definisanje funkcija 2 način: U bloku funkicja je moguće je definisati više funkciju oslanjajući se C# kod definisanja funkcija (blok funkcija)

```
@functions {
  private string dump2(string[] array)
  {
        string s = "";
        foreach (string el in array)
        {
                s += el + (el.Equals(array.Last())?"":", ");
        }
//      for (int i = 0; i < array.Length; i++)
//      {
//              s += array[i] + (i== array.Length-1? "" : "; ");
//      }
        return s;
  }
}
<p>Braća Dalton su: @dump2(daltoni)</p>
```